WEEK-8
Programs using CUDA and CUDA-synchronization

Niharika Gawade
240905284
Roll no:30
CSE-D

1) Implement a program in CUDA which performs convolution operation on one dimensional input array N of size width using a mask array M of size mask_width to produce the resultant one-dimensional array P of size width.

```
#include <stdio.h>
#define MAX 100

__global__ void convolution(int *N, int *M, int *P, int width, int mw) {
   int i = blockIdx.x * blockDim.x + threadIdx.x;
   if (i < width) {
      int sum = 0;
      for (int j = 0; j < mw; j++) {
         int k = i + j - mw / 2;
         if (k >= 0 && k < width)
            sum += N[k] * M[j];
      }
      P[i] = sum;
   }
}

int main() {
   int N[MAX], M[MAX], P[MAX], width, mw;
   int *dN, *dM, *dP;
         printf("Array width and mask width\n");
   scanf("%d%d", &width, &mw);
   printf("Array elements:");
   for (int i = 0; i < width; i++) scanf("%d", &N[i]);
   printf("Mask elements:");
   for (int i = 0; i < mw; i++) scanf("%d", &M[i]);

   cudaMalloc(&dN, width*sizeof(int));
   cudaMalloc(&dM, mw*sizeof(int));
   cudaMalloc(&dP, width*sizeof(int));

   cudaMemcpy(dN,N,width*sizeof(int),cudaMemcpyHostToDevice);
   cudaMemcpy(dM,M,mw*sizeof(int),cudaMemcpyHostToDevice);

   convolution<<<(width+255)/256,256>>>(dN,dM,dP,width,mw);

   cudaMemcpy(P,dP,width*sizeof(int),cudaMemcpyDeviceToHost);

   for (int i = 0; i < width; i++) printf("%d ",P[i]);
```

```
    cudaFree(dN); cudaFree(dM); cudaFree(dP);
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_8$ nvcc q1.cu -o q1
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_8$ ./q1
Array width and mask width
5 3
Array elements:1 2 3 4 5
Mask elements:1 2 5
12 20 28 36 14 5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_8$
```

2) Implement a program in CUDA to perform odd even transposition sort in parallel.

```
#include <stdio.h>
#define MAX 100

__global__ void oddEven(int *a, int n, int phase) {
    int i = 2 * (blockIdx.x * blockDim.x + threadIdx.x) + phase;
    if (i < n-1 && a[i] > a[i+1]) {
        int t=a[i];
        a[i]=a[i+1];
        a[i+1]=t;
    }
}

int main() {
    int a[MAX], n, *d;
    printf("size: ");
    scanf("%d",&n);
    printf("elements: ");
    for(int i=0;i<n;i++) scanf("%d",&a[i]);

    cudaMalloc(&d,n*sizeof(int));
    cudaMemcpy(d,a,n*sizeof(int),cudaMemcpyHostToDevice);

    for(int i=0;i<n;i++) {
        oddEven<<<(n+255)/256,256>>>(d,n,i%2);
        cudaDeviceSynchronize();
    }

    cudaMemcpy(a,d,n*sizeof(int),cudaMemcpyDeviceToHost);

    for(int i=0;i<n;i++) printf("%d ",a[i]);
    cudaFree(d);
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_8$ nvcc q2.cu -o q2
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_8$ ./q2
size: 5
elements: 3 7 4 9 1
1 3 4 7 9 5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_8$
```

3) Implement a program in CUDA to perform parallel rank sort in parallel.

```
#include <stdio.h>
#define MAX 100

__global__ void rankSort(int *a, int *b, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if(i<n) {
        int rank=0;
        for(int j=0;j<n;j++)
            if(a[j]<a[i] || (a[j]==a[i] && j<i))
                rank++;
        b[rank]=a[i];
    }
}

int main() {
    int a[MAX], b[MAX], n, *da, *db;
    printf("size: ");
    scanf("%d",&n);
    printf("elements: ");
    for(int i=0;i<n;i++) scanf("%d",&a[i]);

    cudaMalloc(&da,n*sizeof(int));
    cudaMalloc(&db,n*sizeof(int));

    cudaMemcpy(da,a,n*sizeof(int),cudaMemcpyHostToDevice);

    rankSort<<<(n+255)/256,256>>>(da,db,n);

    cudaMemcpy(b,db,n*sizeof(int),cudaMemcpyDeviceToHost);

    for(int i=0;i<n;i++) printf("%d ",b[i]);

    cudaFree(da); cudaFree(db);
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_8$ nvcc q3.cu -o q3
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_8$ ./q3
size: 6
elements: 7 4 9 0 1 2
0 1 2 4 7 9 5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_8$
```

4) Implement a program in CUDA to count the number of times a given word is repeated in a sentence. (Use Atomic function)

```
#include <stdio.h>
#include <string.h>
#define MAX 1000

__global__ void countWord(char *s, char *w, int *count, int n, int m) {
   int i = blockIdx.x * blockDim.x + threadIdx.x;

   if(i <= n-m) {
      int j;

      for(j=0;j<m;j++)
         if(s[i+j]!=w[j])
            break;

      if(j==m && (i==0 || s[i-1]==' ') &&
        (i+m==n || s[i+m]==' ' || s[i+m]=='\n'))
         atomicAdd(count,1);
   }
}

int main() {
   char s[MAX], w[MAX];
   char *ds,*dw;
   int *count, h=0;

   printf("enter string: ");
   fgets(s,MAX,stdin);

   printf("enter word: ");
   scanf("%s",w);

   int n=strlen(s);
   int m=strlen(w);

   cudaMalloc(&ds,n);
   cudaMalloc(&dw,m);
   cudaMalloc(&count,sizeof(int));

   cudaMemcpy(ds,s,n,cudaMemcpyHostToDevice);
   cudaMemcpy(dw,w,m,cudaMemcpyHostToDevice);
   cudaMemcpy(count,&h,sizeof(int),cudaMemcpyHostToDevice);

   countWord<<<(n+255)/256,256>>>(ds,dw,count,n,m);

   cudaMemcpy(&h,count,sizeof(int),cudaMemcpyDeviceToHost);

   printf("Count = %d\n",h);
```

```
    cudaFree(ds);
    cudaFree(dw);
    cudaFree(count);

    return 0;
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_8$ nvcc q4.cu -o q4
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_8$ ./q4
enter string: what are you are you okay?
enter word: are
Count = 2
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_8$
```

5) Implement a CUDA program that reads a string and replaces every vowel with '*". Use appropriate atomic function to replace the character with *.
Input String S: Parallel CUDA
Output String RS: P*r*ll*1 C*D*

```
#include <stdio.h>
#include <string.h>
#define MAX 1000

__global__ void replaceVowels(int *s, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if(i < n) {
        if(s[i]=='a'||s[i]=='e'||s[i]=='i'||s[i]=='o'||s[i]=='u'||
           s[i]=='A'||s[i]=='E'||s[i]=='I'||s[i]=='O'||s[i]=='U')
            atomicExch(&s[i], '*');
    }
}

int main() {
    char s[MAX];
    int a[MAX];
    int *d;
    int n;

    printf("Enter string: ");
    fgets(s,MAX,stdin);

    n = strlen(s);

    for(int i=0;i<n;i++)
        a[i] = s[i];

    cudaMalloc(&d,n*sizeof(int));

    cudaMemcpy(d,a,n*sizeof(int),cudaMemcpyHostToDevice);
```

```
    replaceVowels<<<(n+255)/256,256>>>(d,n);

    cudaMemcpy(a,d,n*sizeof(int),cudaMemcpy#include <stdio.h>
#include <string.h>
#define MAX 1000

__global__ void replaceVowels(int *s, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if(i < n) {
        if(s[i]=='a'||s[i]=='e'||s[i]=='i'||s[i]=='o'||s[i]=='u'||
           s[i]=='A'||s[i]=='E'||s[i]=='I'||s[i]=='O'||s[i]=='U')
            atomicExch(&s[i], '*');
    }
}

int main() {
    char s[MAX];
    int a[MAX];
    int *d;
    int n;

    printf("Enter string: ");
    fgets(s,MAX,stdin);

    n = strlen(s);

    for(int i=0;i<n;i++)
        a[i] = s[i];

    cudaMalloc(&d,n*sizeof(int));

    cudaMemcpy(d,a,n*sizeof(int),cudaMemcpyHostToDevice);

    replaceVowels<<<(n+255)/256,256>>>(d,n);

    cudaMemcpy(a,d,n*sizeof(int),cudaMemcpyDeviceToHost);

    for(int i=0;i<n;i++)
        printf("%c",(char)a[i]);

    cudaFree(d);

    return 0;
}DeviceToHost);

    for(int i=0;i<n;i++)
        printf("%c",(char)a[i]);

    cudaFree(d);
```

```
    return 0;
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_8$ nvcc q5.cu -o q5
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_8$ ./q5
Enter string: hey how is your day
h*y h*w *s y**r d*y
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_8$
```